Analyse technique — Éditeur de CV avec export PDF compatible ATS

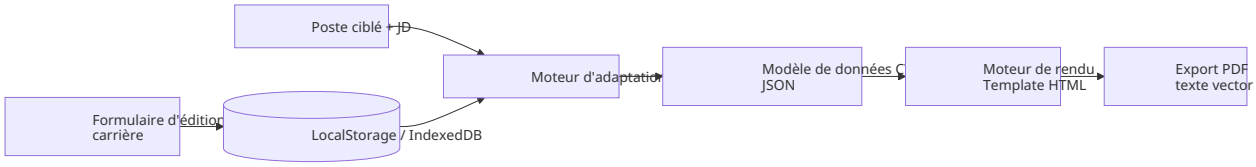
🎯 Cadrage

Besoin : une application où l'utilisateur saisit une fois ses informations de carrière, personnalise la mise en forme de son CV, génère une déclinaison adaptée à un poste ciblé, et exporte un **PDF lisible par les ATS** (Applicant Tracking Systems).

Contraintes fortes :

- Léger (peu de dépendances, chargement rapide)
- **Données 100 % locales** — aucun envoi serveur, pas de compte requis
- Sortie : PDF texte-sélectionnable, structure simple, compatible ATS

🏗️ Architecture recommandée : SPA 100 % locale



Principe clé : séparer données et présentation

Le cœur du produit est un **schéma JSON de CV** (le "source de vérité"). La mise en page n'est que du rendu à partir de ce JSON. C'est ce qui rend facile :

- la personnalisation visuelle (templates) ;
- la génération multi-postes (on part toujours des mêmes données) ;
- l'import/export du fichier utilisateur (portabilité).

```
{
 "profil": { "nom": "", "titre": "", "accroche": "" },
 "contacts": { "email": "", "tel": "", "linkedin": "", "portfolio": "" },
 "experiences": [
 {
 "poste": "", "entreprise": "", "lieu": "",
 "debut": "2024-01", "fin": "2025-02",
 "missions": [""],
 "motsCles": ["C++", "Unreal Engine"]
 }
 ],
 "formations": [], "competences": { "langages": [], "outils": [] },
 "langues": [], "candidatures": [
 { "poste": "", "entreprise": "", "jd": "", "creeLe": "" }
 ]
}
```

⚙️ Choix techniques

Option retenue : Web app statique, zéro backend

Critère	Choix	Justification
Framework	**Aucun, ou Preact/Petite-Vue** (~3–10 Ko)	Un éditeur de formulaires n'a pas besoin de React complet. Vanilla + un petit réactif suffit. Si confort souhaité : **Preact** ou **Svelte** (compilé, runtime minuscule)
Structure	SPA en un bundle unique, hébergeable sur GitHub Pages / Netlify	Coût nul, pas de serveur = pas de fuite possible
Persistance	**localStorage** (simple) ou **IndexedDB** (données lourdes / plusieurs CV)	Clé : versionner le schéma (`schemaVersion: 1`) pour les migrations futures
Import/Export données	Boutons "Exporter mes données (.json)" / "Importer"	localStorage peut être vidé par le navigateur — l'utilisateur doit pouvoir sauvegarder
Éditeur riche	`contenteditable` natif ou champs texte simples	Plus léger = mieux ; des champs texte suffisent pour de l'ATS

Génération du PDF — le point critique

Approche	ATS-safe ?	Poids	Verdict
Impression navigateur (`window.print()` → "Enregistrer en PDF")	✅ Excellente	0 Ko	**Recommandé pour démarrer**
`pdf-lib` ou `jsPDF` (texte natif)	✅ Bonne	~300–500 Ko	Contrôle fin (métadonnées, pagination)
`html2canvas` (capture d'image)	❌ Nulle — texte transformé en pixels, illisible pour un ATS	Lourd	À proscrire

Recommandation en deux temps :

1. **V1 : `window.print()` avec `@media print` CSS** — la stack la plus légère possible, texte nativement vectorisé, sélectionnable, hyperlinks actifs. La personnalisation passe par des variables CSS (couleur, densité, police).
2. **V2 (option) : `pdf-lib`** si besoin de métadonnées PDF propres (titre = "CV Prénom Nom – Poste"), de contrôle de pagination (éviter qu'une expérience soit coupée en deux via `break-inside: avoid`), ou d'un export "silencieux" en un clic.

Règles d'or du PDF compatible ATS

- ❌ Pas de colonnes multiples (les ATS lisent souvent ligne par ligne → texte mélangé) → **une seule colonne** pour la zone d'expérience
- ❌ Pas d'images, icônes, graphiques de compétences, tableaux complexes
- ❌ Pas de texte dans le header/footer de page PDF (souvent ignoré)
- ❌ Pas de polices exotiques ; utiliser **Arial, Helvetica, Calibri, Georgia** (embed obligatoire)
- ✅ Titres de sections standards : "Expériences professionnelles", "Formation", "Compétences"
- ✅ Dates au format cohérent : `01/2024 - 02/2025`

- ✅ Métadonnées du PDF renseignées (titre, auteur) — facile avec pdf-lib
- ✅ Test simple : copier-coller le texte du PDF doit redonner un CV lisible

Le moteur "génération selon le poste ciblé"

C'est la valeur ajoutée différenciante. Pipeline **100 % local, sans IA distante** :

1. **Extraction de mots-clés de la fiche de poste** (JD collée par l'utilisateur) :
   - Tokenisation + suppression des stopwords (liste FR/EN embarquée, ~20 Ko)
   - Rapprochement avec un **glossaire métier** embarqué (extensible via JSON) : "3C" → gameplay, "CI/CD" → dev, etc.
   - Pondération : mots-clés répétés dans la JD = importants
2. **Matching avec le profil** : score de recouvrement entre JD et les `motsCles` / textes des expériences de l'utilisateur
3. **Assistance à la réécriture** (sans LLM distant) :
   - Réordonner compétences et missions selon pertinence
   - Surligner les mots-clés de la JD **absents** du CV → suggestions à ajouter manuellement
   - Générer l'accroche à partir d'un template + mots-clés détectés
4. **Variante optionnelle avec LLM local** (si la machine le permet) : WebLLM en WebGPU — reste 100 % local mais alourdit fortement (plusieurs Go). À considérer comme V3, pas comme cœur du produit.

> 💡 Le niveau 1–3 sans IA couvre déjà 80 % de la valeur ("voici ce que le recruteur/ATS cherche et que ton CV ne contient pas"), en quelques Ko de code.

📦 Découpage en itérations

V0 — Socle

- Formulaire d'édition lié au schéma JSON (identité, contacts, expériences, formations, compétences)
- Persistance localStorage + export/import JSON
- Aperçu live du CV (template ATS-friendly mono-colonne)
- Export PDF via `window.print()` + CSS print

V1 — Personnalisation

- 2–3 templates + palette de couleurs et choix de police (restreints aux polices ATS)
- Réglage densité (marges, taille de texte) et ordre des sections
- Gestion de plusieurs CV à partir du même profil

V2 — Moteur d'adaptation au poste

- Coller une JD → extraction de mots-clés, score de matching
- Suggestions de réordonnancement, surlignage des manques
- Sauvegarde par candidature (poste, entreprise, version du CV)

V3 — Options

- Export `pdf-lib` (métadonnées, contrôle pagination)

- PWA (installation + hors-ligne complet), mode sombre
- Versioning du CV (historique des exports)

🗂️ Structure de code suggérée

```
src/
 schema.ts // types du modèle CV + validation (zod, ~8 Ko)
 storage.ts // localStorage/IndexedDB, migrations
 templates/ // 1 fichier par template (HTML + CSS variables)
 print.css // règles @media print (ATS-safe)
 matcher/ // extraction JD, glossaire, scoring
 glossaire.json // vocabulaire métier FR/EN
```

Dépendances totales (V1) : zod (ou validation maison) + Preact (optionnel). **< 30 Ko gzippé** — l'objectif "léger" est tenu largement.

⚠️ Points de vigilance

1. **localStorage est volatile** : toujours rappeler à l'utilisateur d'exporter ses données ; option File System Access API (Chrome) pour une sauvegarde fichier automatique.
2. **Argument RGPD fort** : pas de serveur = pas de responsable de traitement — "your data never leaves your browser" est un vrai argument marketing.
3. **Compatibilité ATS variable** : les ATS (Taleo, Workday, Greenhouse…) ont des parseurs d'inégale qualité. Mono-colonne + polices standard + sections classiques est le dénominateur commun le plus sûr.
4. **Test ATS** : passer les PDF générés dans des parseurs open source avant chaque release.